# UD 2. Creación de componentes y manejo de datos.

## 1 ¿Qué es un componente en Angular?

Un componente en Angular es la unidad fundamental de construcción de la interfaz de usuario, encapsulando lógica (TypeScript), estructura (HTML) y estilos (CSS) en unidades modulares y reutilizables. Se definen con el decorador **@Component** y gestionan su propia vista y comportamiento.

Los aspectos Clave de los Componentes:

A. **Estructura:** Generalmente se compone de un archivo .ts (lógica), .html (plantilla), .css o .scss (estilos) y .spec.ts (pruebas).

B. **Selector:** Define una etiqueta HTML personalizada (ej. <app-usuario>) para utilizar el componente en la aplicación.

C. **Standalone Components:** Angular utiliza por defecto componentes independientes, que no requieren estar declarados en un NgModule.

D. **Creación:** Se utiliza el CLI con el comando ng generate component nombre-componente.

E. **Comunicación:** Se comunican mediante @Input() (datos de entrada) y @Output() (eventos de salida).

Un componente puede representar desde una página completa hasta un botón pequeño.

## 2 ¿Qué es un componente en Angular?

Aunque puedes crear manualmente cada uno de los archivos necesarios de un componente, no es para nada recomendable. Yo te recomiendo que utilices el CLI de Angular, para ello desde tu terminal escribe el siguiente comando:

```
D:\DWEC\ANGULAR\productos>ng generate component nombre-del-componente
CREATE src/app/nombre-del-componente/nombre-del-componente.component.html (36 bytes)
CREATE src/app/nombre-del-componente/nombre-del-componente.component.spec.ts (652 bytes)
CREATE src/app/nombre-del-componente/nombre-del-componente.component.ts (260 bytes)
CREATE src/app/nombre-del-componente/nombre-del-componente.component.css (0 bytes)
UPDATE src/app/app.module.ts (531 bytes)
```

Esto generará los archivos necesarios para el nuevo componente, incluyendo la plantilla HTML, los estilos CSS y el código TypeScript con su respectiva clase de Tests Unitarios. También declarará el componente en el módulo correspondiente.

```
 Directorio de D:\DWEC\ANGULAR\productos\src\app\nombre-del-componente

19/02/2026  11:30    <DIR>          .
19/02/2026  11:30    <DIR>          ..
19/02/2026  11:30                 0 nombre-del-componente.component.css
19/02/2026  11:30                36 nombre-del-componente.component.html
19/02/2026  11:30               652 nombre-del-componente.component.spec.ts
19/02/2026  11:30               260 nombre-del-componente.component.ts
               4 archivos            948 bytes
               2 dirs  252.152.410.112 bytes libres
```

Un componente en Angular consta de cuatro partes principales:

1. **Plantilla HTML**: Define la estructura y el diseño visual del componente utilizando HTML. Puedes incluir enlaces a propiedades y métodos del componente dentro de la plantilla para mostrar y manipular datos.

2. **Estilos CSS**: Proporciona los estilos visuales para el componente. Puedes utilizar CSS puro o aprovechar las características avanzadas de preprocesadores como Sass o Less.

3. **Pruebas unitarias**: Cuando utilizamos la CLI de angular por defecto nos genera un archivo *.spec.ts el cual contiene una implementación básica de los test unitarios.

4. **Código TypeScript**: Contiene la lógica del componente. Aquí puedes definir propiedades, métodos y eventos del componente. También puedes interactuar con servicios, realizar llamadas a APIs y mucho más.

Dentro del código TypeScript (*.component.ts) encontraremos la anotación @Component la cual se utiliza para decorar una clase y proporcionar metadatos sobre el componente, por defecto se definen las siguientes:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-nombre-del-componente',
  templateUrl: './nombre-del-componente.component.html',
  styleUrls: ['./nombre-del-componente.component.css']
})
export class NombreDelComponenteComponent {

}
```

- **selector**: Especifica el selector CSS que se utilizará para identificar el componente en el HTML. Por ejemplo, si creamos el componente MyComponent` el selector por defecto es 'app-my-component', lo que significa que el componente se puede utilizar en el HTML utilizando la etiqueta <app-my-component>. Cabe destacar que, aunque podemos cambiar el selector, no es muy recomendable ya que rompe la consistencia y la nomenclatura que se recomienda seguir lo que podría confundir a los desarrolladores del proyecto.

- **templateUrl**: Especifica la ruta del archivo de plantilla HTML asociado al componente. En este caso, el archivo de plantilla se encuentra en './my-component.component.html'. Esta plantilla define la estructura y el contenido del componente en HTML.

- **styleUrls**: Especifica un array de rutas de archivos de estilos asociados al componente. En este caso, el archivo de estilos se encuentra en './my-component.component.scss'. Estos estilos se aplicarán al componente y pueden contener reglas CSS para personalizar la apariencia del componente.

Por ejemplo, si tenemos una variable TITLE en el componente:

```
export class NombreDelComponenteComponent {

  title = 'RODRIGO';

}
```

Podemos usar la interpolación para mostrar el valor de TITLE en la plantilla del componente:

```
<H1>Hello, {{title}}</H1>
<p>Felicidades, su aplicación esta funcionando 🎉</p>
```

Cuando la plantilla se renderice, el texto mostrado será "Hola, RODRIGO". La interpolación de variables en Angular permite mantener actualizada la interfaz de usuario a medida que cambian los datos en el componente. Es una forma poderosa y sencilla de mostrar dinámicamente información en las plantillas de Angular.

## 3 Vinculación de datos (Data Binding)

Permite conectar la lógica de la aplicación (componentes, servicios, estado) con la vista (templates HTML) de forma declarativa, clara y eficiente.

Angular ofrece cuatro formas principales de vinculación, cada una pensada para un propósito concreto.

**Interpolación (One-way binding: componente → vista)**

La interpolación permite mostrar valores del componente dentro del HTML.

Sintaxis:

**{{ expresión }}**

Ejemplo:

** Hola {{ nombre }} **

Si nombre cambia en el componente, la vista se actualiza automáticamente y la vista **no** puede modificar el valor.

**Property Binding (One-way binding: componente → vista)**

Permite asignar valores del componente a **propiedades** de elementos HTML o componentes.

Sintaxis:

**[atributo]="expresión"**

Ejemplo:

**<button [disabled]="isLoading">Guardar</button>**

Si isLoading es true, el botón se deshabilita y es más seguro que usar atributos HTML estáticos.

**Event Binding (vista → componente)**

Permite que la vista notifique al componente cuando ocurre un evento.

Sintaxis:

**(evento)="función($event)"**

Ejemplo:

```
<button (click)="guardar()">Guardar</button>
```

El componente recibe el evento y ejecuta la lógica correspondiente

**Two-Way Binding (vista ↔ componente)**

Combina property binding y event binding para mantener sincronizados el modelo y la vista.

En Angular 20, igual que en versiones anteriores, se usa *[(ngModel)]* (requiere importar *FormsModule* ).

Sintaxis:

**[(ngModel)]="propiedad"**

Ejemplo:

```
<input [(ngModel)]="nombre">
<p>Tu nombre es: {{ nombre }} </p>
```

Si el usuario escribe en el input, *nombre* cambia y si *nombre* cambia desde el componente, el input se actualiza.

Desde Angular 16 y consolidado en Angular 20, los **signals** permiten un modelo reactivo más simple y eficiente

Ejemplo con signals (Ts):

```
Import { signal } from '@angular/core'

Export class AppComponent {
   Nombre = signal('Paco');
}
```

En la vista (Html):

```
<p>{{ nombre() }}</p>
<input [value]="nombre()"
(input)="nombre.set($event.target.value)">
```

Las ventajas:

Reactividad más predecible.
Menos dependencias de Zone.js.
Mejor rendimiento.

## 4 Filtros (pipes)

Permiten transformar valores directamente en el template sin modificar el dato original. Angular 20 mantiene el mismo sistema de pipes que versiones anteriores, pero con mejoras de rendimiento gracias al motor de renderizado moderno.

Angular trae varios pipes listos para usar:

| Categoría | Pipes |
|---|---|
| Texto | *uppercase*, *lowcase*, *titlecase* |
| Fechas | *date* |
| Números | *number*, *percent*, *currency* |
| Estructuras | *json*, *slice*, *keyvalue* |
| Async | *async* |

Html

```
<p>{{ nombre |uppercase }}</p>
<p>{{ precio | currency:'EUR' }}</p>
<p>{{ fecha | date:'dd/MM/yyyy' }}</p>
```

**Pipes encadenados**

```
<p>{{ nombre | uppercase  | titlecase }}</p>
```

**Pipes con parámetros**

```
<p>{{ total | number:'1.2-2' }}</p>
```

# 5 Creando nuestros filtros (pipes)

**Pipes un pipe personalizado**

- Generar el pipe

```
Ng generate pipe filter
```

- Implementación del pipe

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'filter'
})
export class FilterPipe implements PipeTransform {

  transform(items: any[], search: string, field: string): any[]
{
    if (!items) return [];
    if (!search) return items;

    search = search.toLowerCase();

    return items.filter(item =>
      item[field].toLowerCase().includes(search)
    );
  }
}
```

- Uso en el template

```
<input type="text" [(ngModel)]="searchText">

<ul>
  <li *ngFor="let alumno of alumnos |
filter:searchText:'nombre'">
    {{ alumno.nombre }}
  </li>
</ul>
```

**Filtrado con Signals**

```
alumnos = signal([
  { nombre: 'Ana' },
  { nombre: 'Luis' },
  { nombre: 'María' }
]);

search = signal('');

filteredAlumnos = computed(() =>
  this.alumnos().filter(a =>

a.nombre.toLowerCase().includes(this.search().toLowerCas
e())
  )
);
```

```
<input [(ngModel)]="search()">

<li *ngFor="let a of filteredAlumnos()">
  {{ a.nombre }}
</li>
```

# 6 Mas sobre componentes

## 6.1 Encapsulación de estilos en componentes

La **encapsulación de estilos** es el mecanismo que usa Angular para controlar **cómo afectan los estilos definidos en un componente** al resto de la aplicación. Su objetivo es evitar conflictos, fugas de estilos y comportamientos inesperados.

Angular 20 mantiene el mismo sistema de encapsulación introducido en Angular 2+, pero con mejoras internas gracias al motor de renderizado moderno.

Sin encapsulación, los estilos de un componente podrían:

- Afectar a otros componentes sin intención
- Ser sobrescritos por estilos globales
- Generar comportamientos inconsistentes en aplicaciones grandes

Angular resuelve esto aplicando **scoping automático** a los estilos del componente.

Angular ofrece **tres modos** principales:

**Encapsulación por defecto**: *Emulated*

Es el método más usado y recomendado. Angular genera los atributos únicos en el HTML del componente.

```
HTML

<div _ngcontent-abc></div>
```

Y reescribe los estilos para que solo afecten a ese componente:

```
CSS

H1[_ngcontent-abc] {
   Color:red
}
```

Las ventajas

- Aísla estilos sin necesidad de Shadow DOM
- Compatible con todos los navegadores
- Fácil de usar

Un ejemplo podría ser:

```
TS

@Component({
  selector: 'app-demo',
  templateUrl: './demo.component.html',
  styleUrls: ['./demo.component.css'],
  encapsulation: ViewEncapsulation.Emulated
})
export class DemoComponent {}
```

**Encapsulación**: *ShadowDom*

Activa el **Shadow DOM nativo**, creando un árbol DOM aislado, las características principales son:

- Estilos totalmente encapsulados
- No se filtran hacia fuera
- No entran estilos globales
- Permite usar *:host*, *::slotted*, etc.

```
TS

@Component({
  selector: 'app-demo',
  templateUrl: './demo.component.html',
  styleUrls: ['./demo.component.css'],
  encapsulation: ViewEncapsulation.ShadowDom
})
export class DemoComponent {}
```

Se puede usar cuando son componentes reutilizables tipo Web Component y con librerías UI que requieren aislamiento total.

**Encapsulación**: *None*

Desactiva la encapsulación: los estilos del componente se vuelven **globales**.

```
TS

@Component({
  selector: 'app-demo',
  templateUrl: './demo.component.html',
  styleUrls: ['./demo.component.css'],
  encapsulation: ViewEncapsulation.None
})
export class DemoComponent {}
```

Se puede usar con Frameworks de diseño (Material, Bootstrap-like), estilos globales compartidos o prototipos rápidos

## 6.2 Ciclo de vida de los componentes

El ciclo de vida de un componente en Angular (incluyendo las últimas versiones) gestiona las etapas desde la creación hasta la destrucción, utilizando "hooks" (**ngOnInit**, **ngOnChanges**, **ngOnDestroy**, etc.) para ejecutar lógica en momentos clave. Los hooks esenciales incluyen inicialización, detección de cambios y limpieza.

Los componentes en Angular:

**constructor():** Primera etapa; instancia la clase. No es un hook de Angular, pero es el inicio.

**ngOnChanges():** Se ejecuta cuando cambia una propiedad de entrada (@Input).

**ngOnInit():** Se ejecuta una vez tras el primer ngOnChanges. Ideal para inicializar datos.

**ngDoCheck():** Detecta cambios que Angular no detecta automáticamente.

**ngAfterContentInit():** Tras proyectar contenido externo (ng-content).

**ngAfterContentChecked():** Tras verificar el contenido proyectado.

**ngAfterViewInit():** Tras inicializar las vistas del componente y sus hijos.

**ngAfterViewChecked():** Tras verificar las vistas del componente y sus hijos.

**ngOnDestroy():** Se ejecuta justo antes de la destrucción del componente; esencial para limpiar suscripciones y evitar fugas de memoria.